MPOWERED

Backend & Database Handover Guide

This guide explains how the MPOWERED backend is structured, how application data is stored in Supabase, and how future developers can maintain, test and rebuild the database.

1. Backend overview

MPOWERED uses Supabase as its backend platform. Supabase provides authentication, PostgreSQL database storage and row-level security for user data.

The frontend communicates with Supabase through service files located in: src/services/

The current backend supports:

- User authentication and session persistence
- User profiles and settings
- Weekly assessment history
- Reflection notes
- Prescriptions
- Support people
- Care Planner appointment plans
- Recording consent and appointment answers

2. Database structure

The main database contains **eight application tables：**

user_profiles： Stores information collected during onboarding, including the user's name, sex, year of birth, diagnosis and health conditions.

user_settings： Stores persistent user preferences such as: Weekly reminders, Appointment, reminders, Text size.

reflection_notes： Stores the user's weekly reflection notes.

Prescriptions： Stores medication information entered in My Health, including medication name, strength, form, dosage and repeat interval.

assessment_cycles： Represents weekly assessment periods. Each user can have one assessment cycle for a particular cycle start date.

assessment_summaries： Stores completed assessment results for: Pain, Movement, Personal care, Social health, Management.

support_people： Stores support contacts saved in Settings. Each contact can include: Name, Phone number, Email, Permission to add questions, Permission to view answers.

appointment_plans: Stores Care Planner appointments. This table contains appointment details together with JSON data for support people, generated questions, consent and answers. Selected question IDs are also stored so an appointment can be restored after logout, login or page refresh.

3. App and database integration

Database operations are separated into service files under src/services.

Important files include:

- userProfile.ts — user profile data
- userSettings.ts — persistent settings
- assessment.ts — assessment cycles and summaries
- reflectionNotes.ts — reflection notes
- prescriptions.ts — prescription CRUD operations
- supportPeople.ts — support people CRUD operations
- appointmentPlans.ts — Care Planner persistence
- supabase.ts — shared Supabase client

4. Authentication and security

Authentication is handled through Supabase Auth. Application tables use user_id to associate records with the authenticated user. Row Level Security (RLS) is enabled on all main backend tables. Policies restrict authenticated users to accessing and modifying their own records. Assessment summaries also verify that the related assessment cycle belongs to the same authenticated user. Foreign keys use ON DELETE CASCADE, so when an Auth user is permanently deleted, the related application records are also removed automatically.

5. Database migrations and local setup

Supabase database definitions are stored in: supabase/migrations/

The initial backend schema is defined in: **20261016081000_initial_backend.sql**

This migration creates the main tables, constraints, foreign keys, validation checks, update triggers, RLS policies and authenticated-user permissions. Later database changes should be added as new migration files rather than editing the production database without recording the change.

To check applied migrations: npx supabase migration list --local

6. Maintaining and testing the backend

When making a database change:

1. Create a new Supabase migration instead of changing only the production schema.
2. Update the relevant service file if the database structure changes.
3. Check any feature context that loads or updates the affected data.
4. Run: npx tsc –noEmit
5. Test the complete user flow in the app.
6. Run the migration against the local Supabase environment before applying it to production.

For CRUD features, confirm that the expected operation occurs:

- Create → INSERT
- Edit → UPDATE
- Remove → DELETE
- Sign in → restore saved data
- Refresh → restore saved data

For authentication testing, Supabase test phone numbers can be reused. A test Auth user can be deleted and registered again with the same test phone number to create a completely new user UUID and clean account state. Before handing over a backend change, confirm that the database migration, service code and app behaviour all match.